

Decentralized Multi-Robot Path Planning with Adaptive Diffusion Convolution Mechanism

Velamala Rahul¹, Deepak Mishra¹, and Sourav Bhowmick^{1,2}

Abstract—Coordinating multiple robots in decentralized path planning scenarios with only local sensing and communication requires careful information exchange. In this connection, learning-based approaches using Graph Neural Networks (GNN) or Graph Convolution Networks (GCN) are promising proponents, but they rely on predefined, fixed communication ranges, also called K -hops. Consequently, this may not be optimal in diverse situations. Effectively, this limitation is addressed in this paper by incorporating a mechanism called Adaptive Diffusion Convolution (ADC) into an established decentralized Multi-Robot Path Planning (MRPP) framework based on diffusion generative models, which are trained with imitation learning. ADC replaces the fixed-hop GNN message passing with a diffusion process whose effective range is automatically learned during the training process. Adaptive control of the information diffusion radius helps the system perform like an expert planner, thereby showing the benefits of adaptive neighbourhood learning in complex multi-robot coordination. Extensive simulation results across varying situations and a case study have corroborated the effectiveness of the proposed methods of the work.

Keywords: Multi-robot path planning, multi-agent systems, adaptive diffusion convolution, graph convolution networks.

I. INTRODUCTION

Efficiently coordinating multiple robots to navigate collision-free in shared environments is one of the major fundamental problems as far as autonomous systems and robotics are concerned. In order for the multiple robot systems to navigate collision-free, MRPP is of utmost importance. Actually, MRPP is the mechanism through which particular trajectory for each robot is planned collision-free with map environment information [1]. Incidentally, MRPP has been studied using a wide range of approaches. Centralized methods, for instance, can find globally optimal solutions using planners such as conflict-based search (CBS) [2] or A* and greedy-search based methods [3]. Also, hybrid A* and various CBS-variants have been used for MRPP in [4]. However, centralized approaches scale poorly with increased computational complexity as the number of robots increases [5]. In order to address this, decentralized methods distribute the decision-making process of the multi-robot systems, offering greater scalability and less computational complexity. Nevertheless, the MRPP problem becomes challenging in a decentralized situation as well. This is mainly

attributed to the fact that the constituent low-end robots operate with limited sensing ranges, restricted communication capabilities, and often even lack access to global positioning systems or a central coordinator [6]. In such scenarios, each robot must rely only on local observations and peer-to-peer communication to make navigation decisions, which requires precise strategies for sharing relevant information to achieve collective goals. Moreover, even though the decentralized approaches are computationally efficient, they often sacrifice global optimality and can be susceptible to deadlocks, i.e., robots blocking each other to follow the desired trajectories without collision [7], [8].

More recently, learning-based policies have emerged to handle the complexities of decentralized coordination. These include methods based on reinforcement learning (RL) [9] and hybrid imitation/RL framework [10]. Another advancement in the related area is the use of GNN to explicitly learn communication and coordination policies from expert demonstrations [11], [12], i.e., synthesizing communication, also called message passing, between one-hop neighbours through GCN and decision-making policies for decentralized MRPP. In particular, the framework proposed by [11] employs a convolutional neural network (CNN) to extract features from local observations and a GNN to model the communication among the robots, and is trained via imitation learning from an expert planner. In addition, [12] proposes a GNN-based approach to compute the edge weights of the neighbor set of the visited vertices, such that the edge weights allow the planner to avoid obstacles and accelerate the search procedure. However, despite the approach being able to learn complex coordination behaviors, the GNN component that depends on a fixed K -hop neighborhood for information aggregation is a bottleneck therein. This is due to the fact that each robot communicates and integrates information only from peers within a predefined number of communication steps (hops). Moreover, another limitation of such approach stem from the fact that the optimal communication range may vary depending on the environment's density, the number of robots, or the specific tactical situation. Consequently, a single fixed K may not be universally optimal, and selecting the best value often requires manual tuning or heuristics, thereby potentially limiting the adaptability and peak performance of the learned policy.

Recently, there has been significant proliferation in the research on diffusion generative models [13] with respect to MRPP [14]–[16]. In [14], diffusion models as priors are learned such that the posterior trajectory distribution is

¹Velamala Rahul, Deepak Mishra, and Sourav Bhowmick are with the Department of Avionics, Indian Institute of Space Science and Technology, Valiamala, Thiruvananthapuram, Kerala-695547, India. E-mails: rahul.velamala@gmail.com, (deepak.mishra, souravbhowmick)@iist.ac.in

²Corresponding author

sampled on task goals subsequently. On the other hand, in [15], single-robot learned diffusion models are coupled with classical search-based techniques to generate the data-driven trajectories of multiple robots under collision constraints. Also, constrained optimization coupled with diffusion models is proposed in [16] to obtain *collision-free, kinematically feasible trajectories* for the multi-robot systems. On the other hand, related to GNN but not related to MRPP problem, graph diffusion convolution (GDC) has been investigated in [17], which extends the message passing in GNN to diffusion process, thereby catering to larger neighbourhood of nodes. However, determining the optimal neighbourhood size for each input graph is computationally expensive in GDC. In order to find a way around this issue, the concept, known as ADC, which uses a learnable diffusion process is proposed in [18] to create an adaptive propagation neighborhood in GNN that automatically learns the optimal neighbourhood size from the given data itself.

Motivated by the above works, in this paper, the fixed-range communication limitation of K -hop neighbours of GNN is addressed by integrating the adaptive mechanism of ADC into the decentralized MRPP framework, enabling the communication range to be learned rather than manually predefined. Actually, The decentralized MRPP framework [11] combines three components: local perception via CNNs, inter-agent communication via GNNs, and policy learning via imitation learning. The readers are referred to [11] and the references therein for these concepts. In particular, ADC replaces the discrete fixed K -hop message passing with a learnable diffusion process, characterized by a continuous radius parameter t . Crucially, this parameter t , representing the effective extent of information propagation, is automatically learned during training, allowing the network itself to determine the most appropriate neighborhood size for feature aggregation based on the task objective. Using imitation learning setup, the network is enabled to adaptively learn the communication range, leading to more effective coordination policies among the robots. The ADC-augmented MRPP architecture is evaluated through simulations across various situations from sparse to denser environments, with performance compared to fixed- K GNN baselines in terms of success rate and path efficiency. The results show that adaptive neighborhood learning improves decentralized MRPP performance by allowing the network to adjust information diffusion based on the coordination task's needs. The following are the contributions of the proposed work:

- ADC-based MRPP mechanism is proposed, which is shown to have better stability compared to fixed- K GNN, and which adaptively learns the neighbourhood communication range in the network, instead of manually predefining it as in the case of fixed- K GNN,
- The continuous radius parameter t is learned that characterizes the learnable diffusion process, such that it represents the effective range of information propagation.

The remainder of the paper is as follows. Section II discusses

the background materials. Sections III and IV describe the proposed methodology and extensive simulation results, respectively, while Section V provides the concluding remarks.

II. BACKGROUND

The following notations are used throughout the paper. Each robot is called an agent. GNN and GCN will be used interchangeably. Let $\mathcal{M} \subset \mathbb{Z}^2$ represent the discrete grid environment of size $W \times H$. The set of agents is $V = \{1, 2, \dots, N\}$. The position of agent i at discrete time step t is $p_i(t) \in \mathcal{M}$. The start and goal positions are $p_i^{(0)}$ and p_i^* , respectively. The set of static obstacle cells is $\mathcal{O} \subset \mathcal{M}$. The local observation (Field-of-View or FOV) for agent i at time t is denoted by

$$\mathbf{Z}_i(t) = [\mathbf{Z}_i^{(1)}(t), \dots, \mathbf{Z}_i^{(C)}(t)], \quad \mathbf{Z}_i^{(c)}(t) \in \mathbb{R}^{H_{\text{FOV}} \times W_{\text{FOV}}}$$

where $C = 3$ and $c \in \{1, 2, 3\}$ represents the number of channels (obstacles/agents, goal, self), and $H_{\text{FOV}}, W_{\text{FOV}}$ are the FOV dimensions. On the other hand, node features for agent i at time t derived from local observations are denoted by $x_i(t) \in \mathbb{R}^{F_{\text{in}}}$, where F_{in} is the dimension of the input feature vector for each agent. The matrix of all node features is $\mathbf{X}_t \in \mathbb{R}^{N \times F_{\text{in}}}$. Hidden features at GNN layer l are $\mathbf{H}^{(l)} \in \mathbb{R}^{N \times F^{(l)}}$, where $F^{(l)}$ is the dimension of the feature vector for each agent at the output of the l^{th} layer. Moreover, the action space for each agent is $\mathcal{U} = \{0, 1, 2, 3, 4\}$ (Idle, Right, Up, Left, Down). The action taken by agent i at time t is $u_i(t)$. The joint action is $\mathbf{u}(t) = (u_1(t), \dots, u_N(t))$. The learned policy is π , that outputs action probabilities or logits. Model parameters are denoted by θ . Euclidean norm of a vector (or a matrix) are represented by $\|\cdot\|$, while $\rho(A)$ represents the spectral radius of matrix A .

Communication among N agents at time t is modeled by a time-varying undirected graph $G_t = (V, E_t)$, where an edge $(i, j) \in E_t$ exists if $\|p_i(t) - p_j(t)\| \leq r_{\text{comm}}$, with r_{comm} being the communication range (radius). The structure of this graph is captured by several key matrices:

- Adjacency Matrix ($\mathbf{A}_t$): An $N \times N$ matrix where $[\mathbf{A}_t]_{ij} = 1$ if $(i, j) \in E_t$ and $i \neq j$, and 0 otherwise.
- Degree Matrix ($\mathbf{D}_t$): A diagonal matrix where $[\mathbf{D}_t]_{ii} = \sum_{j=1}^N [\mathbf{A}_t]_{ij}$.
- We also define $\hat{\mathbf{A}}_t = \mathbf{A}_t + \mathbf{I}$ as the adjacency matrix with self-loops, and $\hat{\mathbf{D}}_t$ as its corresponding degree matrix.

A graph shift operator (GSO), $\mathbf{T}_t$ is a matrix whose sparsity pattern reflects G_t . The GSOs used in this work are the symmetrically normalized adjacency matrix and the normalized graph Laplacian, which are respectively given as

$$\begin{aligned} \tilde{\mathbf{A}}_t &= \hat{\mathbf{D}}_t^{-1/2} \hat{\mathbf{A}}_t \hat{\mathbf{D}}_t^{-1/2}, \\ \mathbf{L}_t &= \mathbf{I} - \hat{\mathbf{D}}_t^{-1/2} \mathbf{A}_t \hat{\mathbf{D}}_t^{-1/2}, \end{aligned} \quad (1)$$

where $\mathbf{I}$ is the identity matrix of appropriate dimension.

III. METHODOLOGY

In order to address the rigidity of the fixed- K GNN approach, the more flexible communication mechanism namely ADC is employed in this work, which is explained as below.

A. ADC Mechanism

The core of ADC is the use of the graph heat kernel, $e^{-t\mathbf{L}}$, as the propagation operator, where $\mathbf{L}$ is a graph Laplacian, e.g., $\mathbf{L}_{\text{ADC}} = \mathbf{I} - \tilde{\mathbf{A}}_t$ and $t \geq 0$ is the diffusion time. The parameter t continuously controls the scale of information aggregation. However, directly computing the matrix exponential is computationally expensive. Therefore, ADC uses a truncated Taylor series to approximate the heat kernel operator. If GSO, $\mathbf{T}_t$ is considered $\tilde{\mathbf{A}}_t$, the propagation operator for the ADC layer at time t , $\mathbf{P}_{\text{ADC}}(t)$ is defined as

$$\mathbf{P}_{\text{ADC}}(t) = \sum_{k=0}^{K_{\text{trunc}}} \theta_k(t) \tilde{\mathbf{A}}_t^k, \quad (2)$$

$$\theta_k(t) = e^{-t} \frac{t^k}{k!},$$

where K_{trunc} is the truncation order that determines the maximum propagation distance considered in the approximation. The crucial part is that the diffusion time t is a learnable parameter, optimized alongside the network weights via backpropagation. This enables the model to adaptively weigh the influence of neighbors at different distances ($k = 0, 1, \dots, K_{\text{trunc}}$) by learning the most effective diffusion scale for the given task.

B. Framework Integration and Operation

The ADC layer replaces the fixed- K GNN in the decentralized MRPP framework. The overall architecture and data flow are illustrated in Figure 1. The local FOV observation is processed by a CNN encoder, and the resulting features are communicated between the nodes via the ADC mechanism. This module uses the current communication graph or GSO ($\mathbf{T}_t$) and learns the optimal diffusion parameter t to control the information propagation range. The decision-making process for each agent i at time t proceeds as follows:

- 1) **Feature Extraction:** The local observation $\mathbf{Z}_i(t)$ is processed by the CNN to yield a feature embedding $x_i(t)$.
- 2) **Adaptive Communication:** The feature embeddings of all agents, $\mathbf{X}_t = [x_1(t), \dots, x_N(t)]^T$, are passed to the ADC layer. Using the GSO, $\mathbf{T}_t$ derived from the current communication graph and the learned parameter t , the ADC layer computes an updated, context-aware output feature vector $\mathbf{h}'_i(t)$ for each agent:

$$\mathbf{H}'(t) = \sigma(\mathbf{P}_{\text{ADC}}(t)\mathbf{X}_t\mathbf{W}_{\text{ADC}}) \quad (3)$$

where $\mathbf{H}'(t)$ is the matrix of output feature vectors, $\mathbf{W}_{\text{ADC}}$ is the learnable weight matrix, and $\sigma(\cdot)$ is the activation function.

- 3) **Action Selection:** The resulting feature vector $\mathbf{h}'_i(t)$ is fed into the final multi layer perceptron (MLP) policy head, which outputs the logits for the discrete action space.

Remark 1: By learning the diffusion parameter t , the proposed framework of MRPP can dynamically adjust its communication strategy, deciding whether to prioritize highly

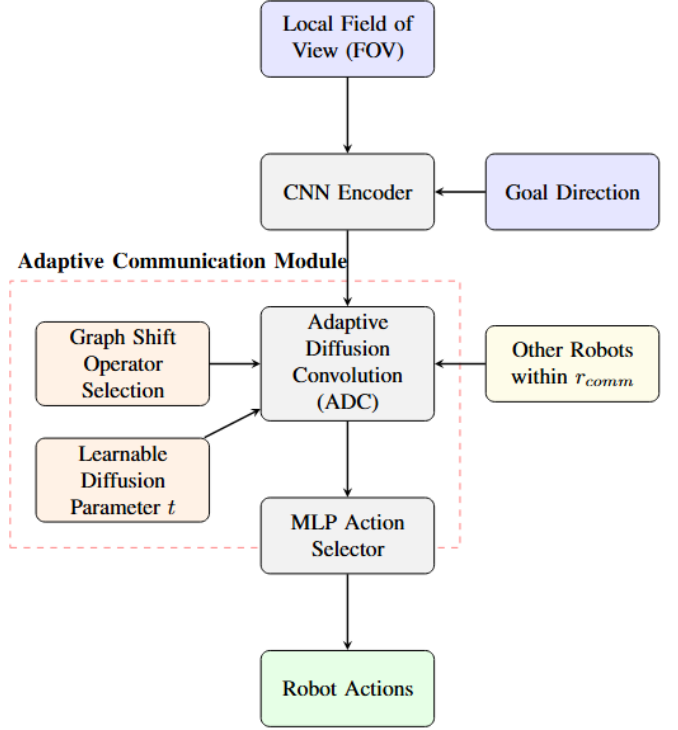


Fig. 1. Architecture of the ADC-augmented MRPP framework

local information or to incorporate signals from more distant robots to resolve complex coordination problems.

C. Comparison Analysis of Fixed- K GNN and ADC Mechanisms

1) **Stability Analysis:** Let $\mathbf{S} \in \mathbb{R}^{N \times N}$ be the normalized GSO with spectral radius, $\rho(\mathbf{S}) \leq 1$. The Laplacian is defined as, $\mathbf{L} = \mathbf{I} - \mathbf{S}$.

Fixed- K GCN Stability: The fixed- K polynomial filter of a K -layer GCN can be written as

$$P_{\text{GCN}} = \sum_{k=0}^K \alpha_k \mathbf{S}^k. \quad (4)$$

Evidently, operator norm admits the bound: $\|P_{\text{GCN}}\| \leq \sum_{k=0}^K |\alpha_k|$. Since the coefficients $\{\alpha_k\}$ are unconstrained during training, therefore $\|P_{\text{GCN}}\|$ may grow arbitrarily large, leading towards instability.

ADC Stability: ADC employs the truncated heat-kernel expansion with learnable diffusion time t , which can be represented by

$$P_{\text{ADC}}(t) = \sum_{k=0}^{K_{\text{trunc}}} \theta_k(t) \mathbf{S}^k, \quad \theta_k(t) = e^{-t} \frac{t^k}{k!}.$$

Proposition 1: If $\rho(\mathbf{S}) \leq 1$, then $\forall t \geq 0$

$$\|P_{\text{ADC}}(t)\| \leq 1. \quad (5)$$

Proof: Since the considered graph G_t is undirected, thus $\mathbf{S}$ is symmetric. Hence $\|\mathbf{S}\| = \rho(\mathbf{S}) \leq 1$ [19]. Thus,

one obtains

$$\|P_{ADC}(t)\| \leq \sum_{k=0}^{K_{\text{trunc}}} \theta_k(t). \quad (6)$$

On the other hand,

$$\sum_{k=0}^{\infty} \theta_k(t) = e^{-t} \sum_{k=0}^{\infty} \frac{t^k}{k!} = e^{-t} e^t = 1. \quad (7)$$

Therefore, $\|P_{ADC}(t)\| \leq 1$. This completes the proof. ■

Remark 2: Proposition 1 shows the better stability and convergent property of ADC in contrast to fixed- K GNN.

2) *Spectral Filtering Characteristics:* For both fixed- K GNN and ADC, each layer's frequency response $h(\lambda)$ is compared on Laplacian eigenmodes.

GCN as Polynomial Filter: When $\mathbf{S} = \mathbf{I} - \mathbf{L}$, the fixed- K GNN operator acts as

$$P_{\text{GCN}}(\mathbf{L}) = \sum_{k=0}^{K-1} \alpha_k (\mathbf{I} - \mathbf{L})^k. \quad (8)$$

Consequently, its spectral response on eigenvalue λ is given as

$$h_{\text{GCN}}(\lambda) = \sum_{k=0}^{K-1} \alpha_k (1 - \lambda)^k, \quad (9)$$

which is a polynomial of degree $K - 1$. The coefficients $\{\alpha_k\}$ give shape flexibility, however the degree is fixed by K .

ADC as Exponential Filter: The ideal (infinite) heat-kernel for ADC is

$$\exp(-t\mathbf{L}) = \sum_{k=0}^{\infty} \frac{(-t\mathbf{L})^k}{k!}, \quad (10)$$

with spectral response

$$h_{\text{ADC}}(\lambda, t) = e^{-t\lambda}. \quad (11)$$

Clearly, $h_{\text{ADC}}(\lambda, t)$ is a continuously-parameterized low-pass filter whose cutoff steepness is controlled by t . Learning t via backpropagation thus adapts the filter bandwidth to the data. On the other hand, since G_t is undirected, thus the associated Laplacian matrix, $\mathbf{L}$ is symmetric, and hence is diagonalizable, such that $\mathbf{L} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$, where $\mathbf{U}$ is an orthogonal matrix. Consequently, $\exp(-t\mathbf{L}) = \mathbf{U} \exp(-t\mathbf{\Lambda}) \mathbf{U}^\top$, and each diagonal entry becomes $e^{-t\lambda_i}$, $i \in \{1, 2, \dots, N\}$. As t increases, higher λ modes are suppressed more, yielding an adaptive low-pass characteristic.

Remark 3: The spectral characteristics of ADC also establishes its adaptive nature of filtering controlled through the parameter t .

IV. SIMULATION RESULTS

A. Experimental Setup

This approach is evaluated on synthetic grid-world environments with the following configuration:

- **Environment size:** 10×10 grid world,
- **Robot count:** 5 robots (5% density),
- **Obstacle density:** Test sets with 10%, 20%, and 30%,
- **Communication radius:** $r_{\text{comm}} = 6$ cells,

- **FOV:** 5×5 cells.

Expert paths have been generated using the CBS method [20]. Moreover, the datasets have been split into training, validation, and test sets using 70:15:15 ratio for three distinct static obstacle densities, with the following number of unique scenarios:

- **10% Obstacle density:** 4767 training, 949 validation, 966 test cases,
- **20% Obstacle density:** 4255 training, 853 validation, 859 test cases,
- **30% Obstacle density:** 2346 training, 471 validation, 482 test cases.

Models have been implemented using PyTorch. Each model has been trained for 100 epochs using the Adam optimizer with a learning rate of 10^{-4} and batch size of 32.

The number of training cases varies with obstacle density, as generating valid and solvable expert trajectories using the CBS planner becomes increasingly computationally expensive in more cluttered environments.

B. Quantitative Performance Metrics

The models have been evaluated using the following metrics on unseen test scenarios:

- **Success Rate (SR):** It is the percentage of tasks where all robots have reached their goals without collisions within the step limit (120 steps).
- **Average Makespan (AM):** It is the average time taken for the last robot to reach its goal in successful episodes. The lower is AM, the better it is.
- **Average Sum of Cost (SOC):** It is the sum of the path lengths of all agents in an episode, averaged over all test cases. This metric measures the total collective effort or cost of a solution. The lower the SOS, the better it is.
- **Number of Parameters (Params):** It is the total count of trainable weights and biases in the model. It indicates model complexity and memory footprint. The lower is better if performance is comparable.

C. Performance and Generalization Analysis

A comparative analysis of the ADC variants and the fixed- K GCN has been performed across varying obstacle densities. Table I, Table II, and Table III show the performance comparison of models trained and evaluated on the dataset with 10%, 20%, and 30% obstacle densities, respectively. This cross-evaluation reveals both performance in matched conditions and generalization to unseen environmental complexities.

1) *Models Trained on 10% Obstacle Density:* Table I shows the performance of models trained on the 10% obstacle dataset. Under the matched test condition (10% density), ADC-FixedT slightly outperforms the best GCN model with respect to SR%. On the other hand, in denser environments, the fixed- K GCNs perform well, but ADC-Main shows strong generalization by matching the best GCN's SR% in the most cluttered 30% density case.

TABLE I

PERFORMANCE OF MODELS TRAINED ON 10% OBSTACLE DENSITY

Test Density	Model	SR (%)	AM	SOC	Params
10%	GCN (K=1)	74.53	10.30	65.54	39.7k
	GCN (K=2)	80.95	10.46	57.47	47.9k
	GCN (K=3)	79.09	10.41	59.98	56.1k
	GCN (K=4)	80.43	10.51	57.88	64.3k
	ADC-Main	75.47	10.34	64.06	39.7k
	ADC-FixedT	81.16	10.55	57.64	39.7k
	ADC-K1	76.50	10.38	63.14	39.7k
20%	GCN (K=1)	50.87	10.39	102.96	39.7k
	GCN (K=2)	57.04	10.73	95.04	47.9k
	GCN (K=3)	58.21	10.77	93.78	56.1k
	GCN (K=4)	56.34	10.69	95.76	64.3k
	ADC-Main	55.88	10.69	96.07	39.7k
	ADC-FixedT	55.53	10.95	97.88	39.7k
	ADC-K1	53.90	10.50	99.19	39.7k
30%	GCN (K=1)	38.38	10.26	132.13	39.7k
	GCN (K=2)	36.93	10.35	134.12	47.9k
	GCN (K=3)	37.34	10.18	133.99	56.1k
	GCN (K=4)	39.42	10.46	131.33	64.3k
	ADC-Main	39.42	10.59	130.01	39.7k
	ADC-FixedT	39.21	10.87	131.99	39.7k
	ADC-K1	35.68	10.17	136.26	39.7k

2) *Models Trained on 20% Obstacle Density:* When trained on the moderately cluttered 20% obstacle dataset, ADC-FixedT performs best, achieving the highest SR % in both the matched 20% test condition (+3.92% over the best GCN) and the sparser 10% test condition (+2.68% improvement), as shown in Table II).

TABLE II

PERFORMANCE OF MODELS TRAINED ON 20% OBSTACLE DENSITY

Test Density	Model	SR (%)	AM	SOC	Params
10%	GCN (K=1)	74.22	10.31	65.94	39.7k
	GCN (K=2)	78.99	10.43	60.64	47.9k
	GCN (K=3)	77.74	10.50	60.97	56.1k
	GCN (K=4)	80.85	10.46	57.72	64.3k
	ADC-Main	77.12	10.25	63.00	39.7k
	ADC-FixedT	83.02	10.57	55.77	39.7k
	ADC-K1	75.88	10.29	64.09	39.7k
20%	GCN (K=1)	55.53	10.61	96.71	39.7k
	GCN (K=2)	62.17	10.94	87.41	47.9k
	GCN (K=3)	61.00	10.94	89.32	56.1k
	GCN (K=4)	61.93	10.87	88.20	64.3k
	ADC-Main	58.56	10.70	92.43	39.7k
	ADC-FixedT	64.61	11.31	85.37	39.7k
	ADC-K1	59.25	10.65	91.30	39.7k
30%	GCN (K=1)	44.40	10.70	123.32	39.7k
	GCN (K=2)	46.68	11.03	121.52	47.9k
	GCN (K=3)	44.19	10.64	124.02	56.1k
	GCN (K=4)	46.89	10.71	120.66	64.3k
	ADC-Main	43.98	10.80	123.69	39.7k
	ADC-FixedT	46.27	11.13	121.43	39.7k
	ADC-K1	45.64	10.74	121.33	39.7k

3) *Models Trained on 30% Obstacle Density:* When trained on the high-density 30% obstacle dataset, ADC-FixedT shows strong generalization to sparser environments, outperforming the best GCNs by +5.22% on 10% and +6.45% on 20% obstacle densities, as shown in Table III. It also slightly outperforms GCNs in the matched 30% test condition, suggesting that the diffusion mechanism learns a more robust communication strategy in complex environmental settings.

TABLE III

PERFORMANCE OF MODELS TRAINED ON 30% OBSTACLE DENSITY

Test Density	Model	SR (%)	AM	SOC	Params
10%	GCN (K=1)	76.40	10.40	62.84	39.7k
	GCN (K=2)	77.02	10.44	62.43	47.9k
	GCN (K=3)	77.12	10.49	62.94	56.1k
	GCN (K=4)	77.43	10.55	61.53	64.3k
	ADC-Main	75.88	10.39	63.52	39.7k
	ADC-FixedT	81.47	10.69	56.70	39.7k
	ADC-K1	76.29	10.42	63.19	39.7k
20%	GCN (K=1)	58.44	10.66	93.39	39.7k
	GCN (K=2)	61.35	10.81	89.62	47.9k
	GCN (K=3)	60.42	11.00	90.74	56.1k
	GCN (K=4)	59.25	10.76	91.75	64.3k
	ADC-Main	59.02	10.72	92.14	39.7k
	ADC-FixedT	65.31	11.25	84.89	39.7k
	ADC-K1	55.41	10.62	97.03	39.7k
30%	GCN (K=1)	43.36	10.73	125.00	39.7k
	GCN (K=2)	44.19	10.85	123.30	47.9k
	GCN (K=3)	46.06	11.24	120.55	56.1k
	GCN (K=4)	47.10	10.59	119.83	64.3k
	ADC-Main	45.85	10.71	121.38	39.7k
	ADC-FixedT	47.51	11.39	119.55	39.7k
	ADC-K1	45.02	10.74	123.00	39.7k

It is to be noted that the ADC-FixedT variant, using a non-learnable diffusion parameter, consistently has emerged as a robust performer. Its success, particularly in generalizing from dense to sparse environments, highlights that the diffusion process itself provides a powerful mechanism for information propagation, even without adaptive learned ‘t’. In contrast, the fully adaptive ADC-Main, while competitive, has not consistently outperformed ADC-FixedT. This suggests that effectively learning the diffusion parameter ‘t’ is a challenging task and may even require more data than available in the smaller, high-density training sets. Evidence for this is seen when training on the largest dataset of 10% obstacle density. It is in this condition that ADC-Main has demonstrated its uniqueness and advantage by matching the top baseline in the most difficult generalization task, which is testing on densest 30% density. This further indicates that with sufficient amount of data, ADC-Main can better leverage its adaptive capability for complex, unseen environmental scenarios.

D. Case Study

Fig. 2 illustrates a 10×10 map with 10% obstacle density where ADC has successfully navigated all robots to their goals while fixed- K GNN ($K = 3$) has failed due to coordination limitations. In fact, the fixed- K GNN approach has created a circular deadlock when robots 1 and 2 simultaneously targeted position (4, 2). Robot 1 (moving westward from (7, 2)) and Robot 2 (moving northward from (4, 1)) both have planned to occupy the same cell, which the $K = 3$ coordination horizon has failed to detect. This has caused Robot 1 to stick at (5, 2) and Robot 2 at (4, 1). Robot 3 subsequently has become trapped at (3, 2), blocked by Robot 2 occupying its goal (4, 1) and Robot 1 blocking its path to (4, 2), creating a three-robot circular dependency. On the other hand, ADC’s adaptive coordination has resolved this scenario through global planning and temporal coordination,

preventing the initial conflict at (4, 2) and enabling all robots to reach their goals.

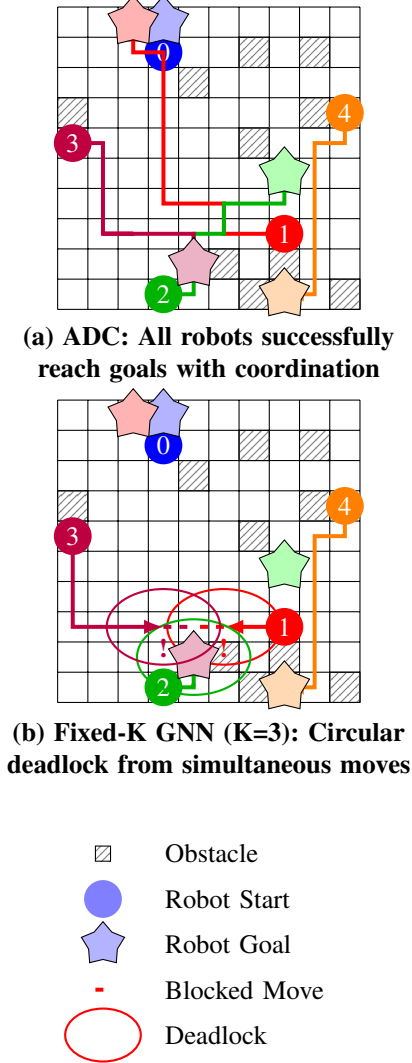


Fig. 2. Multi-robot path finding comparison. (a) ADC coordinates all robots successfully, (b) fixed- K GNN ($K=3$) fails when robots 1 and 2 simultaneously target position (4, 2), creating circular deadlock: Robot 1 stuck at (5, 2), Robot 2 at (4, 1), and Robot 3 at (3, 2)

The related codes are provided in for all the simulation results can be found in the repository link: <https://github.com/rahul-velamala/MAPF-GNN-ADC-1>.

V. CONCLUSION

Through this work, ADC has been introduced into decentralized MRPP, replacing the fixed- K GNN aggregation. Theoretical analysis has established better stability for ADC layers. In addition, extensive simulation experiments across various obstacle densities have confirmed that both diffusion mechanism without learned parameter as well as meachsnim involving learned adaptive communication radius t lead to improved coordination and task success in generalized settings compared to manually fixed communication ranges of GNN. Future work in this direction include formulating algorithms for online multi-robot coverage path planning.

REFERENCES

- [1] P. Surynek, "Bounded sub-optimal multi-robot path planning using satisfiability modulo theory (SMT) approach," in *Proc. IEEE/RSJ IROS, Las Vegas, USA*, October, 2020, pp. 11 631–11 637.
- [2] G. Sharon, R. Stern, A. Felner, and N. R. Sturtevant, "Conflict-based search for optimal multi-agent path finding," *Artificial Intelligence*, vol. 219, pp. 40–66, 2015.
- [3] D. Xiang, H. Lin, J. Ouyang, and D. Huang, "Combined improved A* and greedy algorithm for path planning of multi-objective mobile robot," *Sci. Rep.*, vol. 12, 2022, art. no. 13273.
- [4] Y. Bai, S. Kotpalliwar, C. Kanellakis, and G. Nikolakopoulos, "Multi-agent path planning based on conflict-based search (CBS) variations for heterogeneous robots," *J. Intell. Robot. Syst.*, vol. 111, 2025, art. no. 26.
- [5] Y. Bi, J. Luo, J. Zhu, J. Liu, and W. Li, "Decentralized multi-robot navigation based on deep reinforcement learning and trajectory optimization," *Biomimetics*, vol. 10, 2025, art. no. 366.
- [6] L. Maignon, L. Jeanpierre, and A.-I. Mouaddib, "Coordinated multi robot exploration under communication constraints using decentralized Markov decision processes," in *Proc. AAAI Conf. Artificial Intelligence, Toronto, Canada*, July, 2012, pp. 2017–2023.
- [7] R. Zheng and S. Li, "MCCA: A decentralized method for collision and deadlock avoidance with nonholonomic robots," *IEEE Robot. Autom. Lett.*, vol. 9, no. 3, pp. 2710–2717, 2024.
- [8] H. Li, T. Zhao, and S. Dian, "Prioritized planning algorithm for multi-robot collision avoidance based on artificial untraversable vertex," *Appl. Intell.*, vol. 52, p. 429–451, 2022.
- [9] M. Everett, Y. F. Chen, and J. P. How, "Motion planning among dynamic, decision-making agents with deep reinforcement learning," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 3052–3059.
- [10] G. Sartoretti, J. Kerr, Y. Shi, G. Wagner, T. K. S. Kumar, S. Koenig, and H. Choset, "PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning," *IEEE Robot. Automat. Lett.*, vol. 4, no. 3, pp. 2378–2385, 2019.
- [11] Q. Li, F. Gama, A. Ribeiro, and A. Prorok, "Graph neural networks for decentralized multi robot path planning," in *Proc. IEEE/RSJ IROS, Las Vegas, USA*, October, 2020, pp. 11 785–11 792.
- [12] X. Diao, W. Chi, and J. Wang, "Graph neural network based method for robot path planning," *Biomim. Intell. Robotics*, vol. 4, no. 1, 2024, art. no. 100147.
- [13] L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui, and M.-H. Yang, "Diffusion models: A comprehensive survey of methods and applications," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–39, 2023, art. no. 105.
- [14] J. Carvalho, A. T. Le, M. Baier, D. Koert, and J. Peters, "Motion planning diffusion: Learning and planning of robot motions with diffusion models," in *Proc. IEEE/RSJ IROS, Detroit, USA*, October, 2023, pp. 1916–1923.
- [15] Y. Shaoul, I. Mishani, S. Vats, J. Li, and M. Likhachev, "Multi-robot motion planning with diffusion models," in *Proc. 13th ICLR, Singapore*, April, 2025, in print.
- [16] J. Liang, J. K. Christopher, S. Koenig, and F. Fioretto, "Simultaneous multi-robot motion planning with projected diffusion models," in *Proc. 42nd ICML, Vancouver, Canada*, July, 2025, in print.
- [17] J. Gasteiger, S. Weissenberger, and S. Günnemann, "Diffusion improves graph learning," in *Proc. 33rd NeurIPS, Vancouver, Canada*, December, 2019, pp. 13 366–13 378.
- [18] J. Zhao, Y. Dong, M. Ding, E. Kharlamov, and J. Tang, "Adaptive diffusion in graph neural networks," in *Proc. 35th NeurIPS, Canada*, December, 2021, pp. 23 321–23 333.
- [19] R. S. Varga, *Matrix Iterative Analysis*, 2nd ed. Springer, 2009.
- [20] G. Sharon, R. Stern, A. Felner, and N. Sturtevant, "Conflict-based search for optimal multi-agent pathfinding," *Artif. Intell.*, vol. 219, pp. 40–66, 2015.